

EventRouting

Ereignisse (Events)

- Delegate-Eigenschaften, welche bei Control-spezifischen Triggern ausgelöst wird (z.B. Button-Click)
- Werden auch über Attribute in XAML gesetzt
- Syntax
 - Eventname = „EventHandlerMethodenName“
`<Button Click="Button_Click">Btn</Button>`
- Event-Handler muss im Code-Behind vorhanden sein
 - Visual Studio und Blend erstellen die Methode automatisch

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    //C#-Code, welcher beim Auslösen des Events ausgeführt werden soll
}
```
- WPF unterscheidet zwischen ungeleiteten/unrouted (Direct) und geleiteten/routed (Bubble/Tunnel) Events

Events ermöglichen die Kommunikation und Interaktion zwischen Elementen in der Benutzeroberfläche und dem Rest des Programms. Ein Event wird ausgelöst, um auf eine Benutzeraktion oder ein Systemereignis zu reagieren und liegt in der Klasse als Delegate-Eigenschaft vor. Es gibt sowohl allgemeine Events, welche in nahezu allen Controls vorhanden sind, als auch Control-spezifische Events.

Ein Eventhandler ist eine Methode, die aufgerufen wird, wenn ein Event eintritt. Der Eventhandler wird im CodeBehind registriert, um den Code zu definieren, der ausgeführt werden soll. Wenn ein Event ausgelöst wird, ruft es den registrierten Eventhandler auf, der die entsprechende Logik ausführt.

WPF unterscheidet zwischen geleiteten (routed) Events, welche es in die zwei Kategorien (Richtungen) unterteilt und Bubbleing- bzw Tunneling-Events nennt, und ungeleitete (unrouted) Events, sogenannte Direkt-Events. Letztere sind insbesondere sogenannte PropertyChanged-Events, welche auf Veränderungen von Control-spezifischen Properties reagieren, als auch z.B. das Click-Event des Buttons.

Eine Beschreibung der routed Events folgt auf den folgenden Seiten.

Basisklassen-Events

- UI-Elemente erhalten zusätzliche Events als AttachedProperties von anderen Basisklassen
 - z.B. ButtonBase.Click, TextBoxBase.TextChanged
- EventHandler triggern beim Auslösen jedes entsprechenden Events innerhalb des UI-Elements

```
<Window x:Name="Wnd_Main"
        ButtonBase.Click="Wnd_Main_Click"
        TextBoxBase.TextChanged="Wnd_Main_TextChanged">
  <StackPanel>
    <Button Content="ButtonBase-Test"/>
    <TextBox Text="TextBoxBase-Test"/>
  </StackPanel>
</Window>
```

Bestimmte häufig verwendete Direkt-Events können als Attached Properties der Basisklassen der entsprechenden Controls in übergeordneten Elementen definiert werden. Diese werden dann zusätzlich zu den spezifischen Events ausgelöst. So können die Aktionen auf einer globaleren Ebene abgefangen und auf diese reagiert werden.

Visual vs. Logical Tree

- Element Tree setzt sich zusammen aus Visual Tree und Logical Tree
- Logical Tree ist der logische Aufbau einer XAML-Datei
 - entspricht im Grunde der Baumstruktur des XAML-Code im Editor
 - Enthält auch nicht-grafische Elemente, wie z.B. Converter
- Visual Tree
 - Enthält alle zu rendernden Elemente
 - Detailliertere Aufsplitterung der sichtbaren Elemente des Logical Tree
- Ohne Eventrouting, keine Eventbehandlung komplexer, modularer Steuerelemente -> Bubbling/ Tunneling

In WPF gibt es zwei wichtige parallele Konzepte zur Strukturierung der Elemente in der Benutzeroberfläche: den Visual Tree (Visueller Baum) und den Logical Tree (Logischer Baum).

Der Logical Tree repräsentiert die logische Hierarchie der Elemente in der Benutzeroberfläche. Er beschreibt die funktionalen Beziehungen zwischen den Elementen unabhängig von ihrer visuellen Darstellung. Dabei haben die Elemente ein übergeordnetes Element (Parent) und können mehrere untergeordnete Elemente (Children) haben. Der Logical Tree dient dazu, die logische Struktur der Benutzeroberfläche zu repräsentieren, z.B. die Beziehung zwischen einem Fenster und seinen enthaltenen Steuerelementen.

Der Visual Tree hingegen repräsentiert die visuelle Hierarchie der sichtbaren Elemente in der Benutzeroberfläche. Jedes sichtbare Element wird durch ein visuelles Objekt im Visual Tree dargestellt. Der Visual Tree beschreibt die Beziehung zwischen den Elementen in Bezug auf deren visuelle Anordnung und Darstellung. Er wird verwendet, um die Positionierung, Größe und Darstellung der Elemente zu bestimmen. Dabei besteht ein Element, das im Logical Tree eine Einheit ist im Visual Tree für gewöhnlich aus mehreren Elementen, da hier sämtliche zu rendernden Bestandteile abgebildet werden.

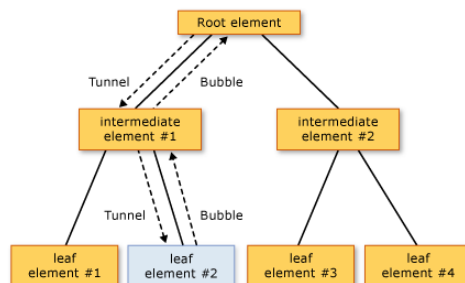
Der Visual Tree und der Logical Tree können miteinander verbunden sein, aber sie sind nicht immer identisch. Manchmal gibt es logische Elemente ohne visuelle Darstellung im Visual Tree, wie z.B. Data-Bindings oder Ressourcen. In einigen Fällen kann ein einzelnes logisches Element mehrere visuelle Darstellungen haben.

Eventrouting

- Tunneling (*Preview-Events*)
 - EventHandler wird beim Root-Element aufgerufen
 - Event wandert durch die Hierarchie bis zum Auslöser
- Bubbling
 - EventHandler wird beim Auslöser aufgerufen
 - Ereignis wandet durch die Hierarchie bis zum Root-Element
- Unterbrechung des Routings:


```
e.Handled = true
```
- Originalquelle im VisualTree:


```
e.OriginalSource
```



Das Event-Routing in WPF ermöglicht die Weiterleitung von Ereignissen von einem Element zum anderen in der visuellen Hierarchie der Benutzeroberfläche. Dabei gibt es zwei Arten des Event-Routings: Tunneling und Bubbling, welche immer als Paar mit dem selben Trigger auftauchen. Kommt es zu Auslösung, wird immer zuerst das Tunneling- und dann das Bubbling-Event ausgeführt.

Beim Tunneling wird das Ereignis vom übergeordneten Element zum untergeordneten Element weitergeleitet. Das Ereignis durchläuft den Visual Tree in der Reihenfolge von oben nach unten. Dies ermöglicht es übergeordneten Elementen, das Ereignis vor der eigentlichen Behandlung zu verarbeiten oder zu modifizieren. Tunneling-Events sind durch das Präfix "Preview" gekennzeichnet.

Beim Bubbling wird das Ereignis vom untergeordneten Element zum übergeordneten Element weitergeleitet. Das Ereignis durchläuft den Visual Tree in der Reihenfolge von unten nach oben. Es gibt keine spezielle Kennzeichnung, außer dass diese Events genauso heißen, wie das entsprechende Tunneling-Event, nur ohne das Präfix.

Während des Event-Routings kann ein Ereignis über die entsprechende Eigenschaft des EventArgs-Objekt als "Handled" markiert werden. Dadurch wird verhindert, dass das Ereignis weiter im Event-Routing propagiert wird. Wenn ein Ereignis als "Handled" markiert ist, werden keine weiteren Eventhandler aufgerufen.

Die Eigenschaft "OriginalSource" gibt das ursprüngliche Element zurück, auf dem das Ereignis ausgelöst wurde, unabhängig davon, welches Element das Ereignis tatsächlich behandelt.